

Assignment 2

SEN381 — Software Engineering | 2026

Group: Group I

Participating Members:

- Gerald Enright (577830)
- Keletso Marota (601632)
- Mogau Malope (600192)

Task 1

Design Problem

CivicConnect has a controlled service-request lifecycle. A request may be submitted, assigned or accepted, updated, rejected, resolved and closed, with different actions being appropriate at different stages.

This is a problem because implementation could have the request status represented as an enumeration (enum) alongside methods that are based on the enum values. For small simple transitions this may be reasonable. However, the design becomes difficult if multiple behaviors depend on the current status. If those rules become distributed across multiple methods containing repeated status checks, changes to the lifecycle can require changes in several locations.

Coupling and cohesion.

Coupling is the degree of interdependence between modules or classes: a module is coupled to another if a change in one forces a change in the other. Cohesion concerns how closely related the responsibilities within a software element are. In the case for CivicConnect coupling will be determined by how many parts of the system need to know the detailed rules associated with every request status. While cohesion will be determined by whether each component contains their own state-specific behavior rules. This is important to remember as development continues, because a system that is over-coupled will be difficult to test or fix should there be an issue. While a system that has low cohesion will be difficult to maintain in the long run, because behavior rules are scattered between components.

Relevant SOLID principles.

- **Single Responsibility Principle (SRP):** A class should have only one reason to change (Stevenson; 2018). When a class mixes business logic with unrelated concerns, it has more than one reason to change and cohesion can decline. Therefore state-specific behavior should be separated to provide a way of keeping individual components more focused. However this does not mean that the class should be responsible for only one thing.
- **Open/Closed Principle (OCP):** Software entities should be open for extension but closed for modification (Plösch; 2016). A design that must be edited every time a new variant is introduced violates OCP and increases regression risk on code that was already tested.
- **Liskov Substitution Principle (LSP):** Subtypes should be usable wherever their base type is expected without breaking the program's correctness. Using different types of service requests or user roles represented through inheritance, each subtype could honour behavior expected from the parent abstraction.
- **Interface Segregation Principle (ISP):** Clients should not be forced to depend on methods they do not need. Instead of creating one large interface to contain many operations simultaneously, focus should be switched to smaller focus groups.

- **Dependency Inversion Principle (DIP):** High-level modules should depend on abstractions rather than concrete low-level implementations. The request-management logic could depend on an abstracted component rather than directly depending on email or SMS implementation.

Problems Identified

- **Problem 1: Reacting to a service-request state change.** When the state of a request is changed several independent things should happen: The requester needs feedback, a history entry should be created, and staff views should reflect the change. If an individual class handles all these steps for its own status update method then the class takes on more responsibilities than what was originally intended and every new reaction will require editing and re-testing the already working code (Tiwari; 2018).
- **Problem 2: Creating and validating category-specific requests.** There should be a supported, controlled category mechanism and each category plausibly needs different fields and validation. If this is created as one creation routine containing a chain of decision statements keyed on category, the routine must be reopened every time a category is added or has a rule change, and a change aimed at one category risks breaking another - a direct OCP violation.

Alternatives For Each Problem

Literature emphasizes that design patterns should be selected according to the characteristics of the problem rather than applied automatically (Naghdipour; 2023). There are also situations identified where pattern structures can introduce undesirable design smells, demonstrating that the use of a pattern does not guarantee a better design (Almadi; 2021).

Problem 1

- **Alternative A: Centralized conditional approach**

A straightforward solution is to represent the request state using an enumeration or similar value and keep lifecycle behavior within a service or request-management. In essence this means that each service will get a status value from an enum and the service handler will manage what needs to be done based on the status value. This approach is most appropriate when the lifecycle is small, stable and straightforward.

Advantages	Limitations
The main advantage is simplicity, as there are few classes and relationships, making the initial design easier to understand.	The main risk is that the central service grows as state-dependant behavior increases.
Also makes control flow explicit where a developer investigating a lifecycle operation can find all the relevant rules in one location.	As additional states and operations are introduced the conditional logic can become complicated, where changes to one state may require modifying a singular large file that also handles rules for other states.
Testing can also be straightforward when the number of states and transitions is small, by providing a request state and verifying if the operation is accepted or not.	Cohesion can be reduced as changes to one lifecycle are likely to affect unrelated behavior.

- **Alternative B: State-Oriented design**

Another solution is to represent the behavior associated with each lifecycle state separately. This means that the service will still get a status value from an enum and the service handler will call the request rules from a separately stored file for each state. This approach is particularly relevant when the behavior of the request changes substantially according to its state.

Research on state-based software design demonstrates that state-oriented representations can explicitly associate responsibilities and transitions with states, supporting the separation of state-specific behavior (Nikitin; 2023).

Advantages	Limitations
The main advantage is the separation of state-dependant behavior. Each state can have a relatively focused responsibility, reducing the amount of state specific logic that a central component needs to maintain.	The main disadvantage is the additional complexity, as the system would move away from one service containing all lifecycle rules to a system with state abstraction, multiple concrete state classes, relationships between requests and the current state, transition logic, and additional tests and documentation.
This also makes introducing more lifecycle states a simple matter as new behavior can be introduced in an isolated manner rather than in an increasingly large conditional structure.	There is a risk as development needs to understand and maintain a complex architecture, where responsibilities are technically separated but unnecessary abstraction may reduce practical maintainability.
The design also improves test isolation as individual state behaviors can be tested separately.	

Comparison of the Alternatives (Problem 1)

Criterion	Centralized Conditionals	State-oriented design
Initial simplicity	HIGH	MODERATE/LOW
Number of classes	LOW	HIGH
Lifecycle behavior cohesion	Can decrease as complexity grows	Potentially High
Coupling	Initially Low	More relationships between state objects
Extensibility	Can become difficult as states increase	Better suited for evolving state behavior
Testability	Straightforward for small lifecycle	Can isolate individual state behaviors
Understandability	Good when lifecycle is simple	Good when lifecycle is complex, but more concepts need to be understood
Risk of over-engineering	LOW	HIGH
Suitable when	Few stable states and simple rules	Many states or substantial state specific behavior

Problem 2

• Alternative A: Factory-Based Request Creation

The factory-based approach encapsulates the object-creation decision so that callers do not need to know how a particular category request is constructed. This approach lets the caller interact with the factory mechanism that handles the category rather than calling a category-specific constructor. Particularly useful in situations where categories represent genuinely different request types with different construction requirements.

Advantages	Limitations
The primary advantage is the simplicity of responsibility as the factory approach provides a clear boundary around object creation, where the request-management component does not need to know the construction details of every request type.	The OCP problem is not necessarily solved in the factory based approach. A poorly designed system would just be moving the original conditional chain to a new class, here the conditional decision logic still has to be modified every time a category is added. Thus the design would just relocate the problem and not solve it.
Design becomes easier to understand than introducing separate behavioral abstraction when the main variation is simply which type of request should be constructed.	The factory approach is strongest when the main variation focus is on object construction. Further separation may be required in situations where each category also has independently evolving validation and business rules.
Testing object creation becomes more focused as construction logic is isolated from the calling service.	

• **Alternative B: Strategy-Based Category Handlers**

This approach focuses on encapsulating category-specific creation and validation behavior behind common abstraction, with each category having a handler that contains the rules specific to that category, and a central request-creation process that coordinates the operation rather than containing the validation and creation rules for each category.

Advantages	Limitations
The strongest advantage is the improved extensibility where a new category can potentially be implemented by adding another handler rather than adding another branch to an already existing creation method.	The main limitation is the additional abstraction, where instead of one creation routine the system could require a handler interface, multiple concrete handlers, a mechanism for selecting the correct handler, and additional classes and tests. Which will make the design more difficult to understand.
Category-specific behavior is also more cohesive because the validation and creation rules for a category are located together.	There is also an important empirical reason not to assume that Strategy is automatically better. Silva (2021) conducted a quasi-experiment examining the impact of the Strategy Pattern on maintainability. Their findings indicate that the introduction of Strategy can itself affect maintenance performance, reinforcing the need to consider whether the additional abstraction is justified in the particular context.
Testing can become more focused, as a test can focus on a specific category handler rather than executing a large central creation routine containing rules for every category.	
There is also more support for categories to evolve differently. If one category develops significantly more complex validation rules than another, the complexity will remain in the relevant handler instead of increasing the complexity of every request-creation operation.	

Comparison of the Alternatives (Problem 2)

Criterion	Factory	Strategy
Primary purpose	Encapsulate object creation	Encapsulate varying behavior

Criterion	Factory	Strategy
Best fit	Different request types require different construction	Categories have different validation/business behavior
Creation responsibility	Strong	Moderate/Strong
Category-specific validation	Possible, but may require separate validators	Strong
OCP support	Depends on implementation	Strong
Initial simplicity	High	Low
Additional abstractions	Moderate	High
Test Isolation	Good	Good
Risk of over-engineering	Low	High
Best Use	Categories represent different request objects	Categories have independently evolving rules

Recommendation feeding M2

Research: Design patterns should be selected according to the actual characteristics of the design problem rather than because the pattern is a recognized solution. Naghdipour (2023) conducted a systematic literature review of approaches to software design-pattern selection and identified pattern selection as a non-trivial decision problem. There is also evidence that pattern structure and implementation can influence software quality, meaning that the use of a pattern alone is not evidence of improved maintainability (Wedyan; 2020). This supports evaluating candidate solutions against the characteristics of the particular design problem rather than selecting a pattern simply because it is available.

- **Problem 1 Recommendation:** A State-oriented design should be considered in the situation where service-request operations contain substantial state-dependant behavior and the lifecycle is likely to evolve. A Centralized conditional approach should be considered in the situation where there are a small number of stable states and straightforward transition rules.
- **Problem 2 Recommendation:** A strategy-based approach is recommended in the event that the categories are determined to have substantially different and evolving validation or business rules. In the scenario where the categories primarily represent different request types with different construction requirements, a factory-based approach is recommended.

Task 2

Persistence and Data-Integrity Decisions

Using PostgreSQL, the team's selected database, this part investigates how CivicConnect should maintain data accuracy when a step fails or two persons act simultaneously.

2.1 Chosen operation: assigning a request

When a staff member accepts a request, five things are saved together: the owner (FR-011), the status change from Received to Assigned (FR-012), a history entry (FR-013), an audit record (NFR-007) and a message for the requester (FR-007). Three things can go wrong:

- **Partial save:** the status changes but the history entry is lost (NFR-007).

- **Two staff at once:** both assign the same request and the last save wins (NFR-005).
- **Rules bypassed:** a script saves an invalid change, e.g. Received → Closed (FR-012).

2.2 Research into correctness mechanisms

Transactions

The software must determine which of the multiple changes made during a transaction belong inside it. To ensure that nothing is half-saved and that any failure causes ROLLBACK, the status/owner check and all five saves should share a single, brief PostgreSQL transaction. The user's decision-making time and queue loading remain outside of the transaction. CivicConnect has a single database and backend, therefore one local transaction is sufficient (D-001, D-002).

Where rules are checked — Who checks what

Rule	UI	Backend	PostgreSQL
Only staff may assign	Hide button	Main check	–
Status change is valid	Show valid actions	Main check	CHECK on status values
Request not already owned	Show owner	Clear message	Conditional UPDATE
History cannot be changed	–	No edit feature	INSERT/SELECT rights only

It is dangerous to have just one layer check the rules. Bypassing the user interface is possible, incorrect data from scripts cannot be prevented by the backend alone, and rules stored just in the database are difficult to test and may result in ambiguous errors. As a result, each layer has a specific function: PostgreSQL safeguards the stored data and prevents incorrect entries, the backend verifies the business rules, and the user interface helps users submit accurate information.

Concurrency

Under PostgreSQL's default Read Committed level, two staff can both read a request as unassigned and both save, so one update is silently lost (PostgreSQL Global Development Group, 2026a).

Ways to prevent a lost update

Option	How it works	Trade-off
Conditional UPDATE	e.g. Update WHERE assigned_staff_id IS NULL; 0 rows changed means someone was first	Simple, no locks. Rule repeated in SQL
Version number	Save only if the row's version is unchanged	Works for any edit. User must retry
SELECT ... FOR UPDATE	Lock the row before checking	No clash; others wait; deadlock risk
Serializable	PostgreSQL cancels one clashing transaction	No manual check; backend must retry

Caching

Cache needed to determine an assignment should never be stored because it may speed up the staff queue (NFR-004) but may display outdated data (Microsoft, 2025). M2 doesn't require a cache; instead, indexes should be tried first, and caching should only be reviewed if NFR-004 is not satisfied. As long as the cache is emptied each time an administrator makes changes, the category list is the safest later option because it rarely changes.

2.3 Alternative approaches compared

A: The backend checks rules and saves each change separately.

B: A PostgreSQL stored procedure does everything.

C: Layered: backend rules run in one transaction, with PostgreSQL constraints and a conditional UPDATE as safety nets.

Criterion	A	B	C
All-or-nothing saves	Weak	Strong	Strong
Two staff at once	Weak	Strong	Strong
Protection from scripts	Weak	Strong	Good
Easy to read and test	Strong	Weak	Strong
Clear messages for users	Medium	Weak	Strong
Effort for the team	Low	High	Medium

A is the quickest to build but fails NFR-005 and NFR-007. B is strong, but it hides business rules in SQL that the team would find hard to review and test. C takes a little more effort, but each layer has a clear job, so **C is the best fit for CivicConnect**.

2.4 Recommendation for the project

The team recommends the following (final decision in M2):

ID	Recommendation	Feeds into
R2-1	One PostgreSQL transaction for all five saves, run by the backend	New ADR; new risk: partial save; M3 rollback test
R2-2	Conditional UPDATE to claim a request; version column for other edits	M2 data model; new risk: double ownership; M3 concurrency test
R2-3	Backend owns role and status rules; PostgreSQL constraints; insert-only history	M2 data model; RTM links for FR-011–FR-013, NFR-005, NFR-007
R2-4	No caching in M2; indexes first	NFR-004 performance check

Limitations: This assumes only a few staff act on the same request at once; if clashes turn out to be common, row locking may be a better choice. The design also depends on the reassignment rules (AC-011.2), which are still an open M1 item and would change the UPDATE condition.

Task 3

APIs and Integration Decisions

3.1 Integration Requirement

The existing architecture separates the user interface, backend/API and persistence layers, as established by D-001 and D-002. An integration mechanism is therefore required between the Staff Operations Panel and the Requester Portal when a service request changes state.

The specific case considered is the transition of a request's status by Service Staff, as defined by FR-012. NFR-002 requires the updated status to reach the Requester Portal within 15 seconds. At the same time, NFR-003 requires that information belonging to another requester not be exposed, while NFR-005 and FR-011 require controls to prevent two staff members from owning the same request.

The Staff Operations Panel acts as the producer by submitting changes through the backend, while the Requester Portal acts as the consumer by retrieving the updated request state. Without a defined integration approach, individual screens could implement different endpoints and validation rules. This could result in stale request statuses, which would conflict with NFR-002, as well as duplicated business logic across the user interfaces. Centralising these rules through the backend therefore supports the purpose of D-002 and provides a more consistent approach to request processing.

3.2 Comparison of Integration Mechanisms

Since both clients are browser-based, HTTP will form the basis of communication. The main decision is therefore how the Requester Portal should become aware of a change that it did not initiate.

Criteria	REST with Client Polling	REST with Push (WebSocket/SSE)
Coupling	Low. The client periodically requests the latest state.	Slightly higher because a persistent connection or subscription is required.
NFR-002	Can be satisfied with a polling interval of approximately 10 seconds or less, although this generates additional requests.	Can be satisfied with near-instant updates.
Failure behaviour	Degrades gradually. A missed poll results in a temporarily stale view until the next request.	A dropped connection requires reconnection or fallback logic, otherwise the client may become stale.
Testability	Simple request/response behaviour is straightforward to test.	More complex because connection and lifecycle behaviour must also be tested.
Team unfamiliarity risk (RSK-001)	Low because REST is widely understood and uses established patterns.	Higher because real-time communication introduces additional infrastructure and development complexity.

Where REST is used, the API should follow basic resource and method conventions. Resource-oriented nouns should be used for URIs, HTTP verbs should represent operations, and requests should remain stateless by carrying the information required for processing rather than depending on server-side session state. A service becomes more RESTful as it makes greater use of appropriate URIs, HTTP methods and hypermedia rather than relying on a single tunnelled endpoint.

Full HATEOAS maturity is not required for an internal API of this size. The additional discoverability and self-description provided by higher Richardson Maturity Model levels would provide limited value while the system has only a small number of functional requirements and no external or third-party consumers. Implementing these levels at this stage would therefore add complexity without a proportionate benefit (Devopedia, 2020; RESTful API, n.d.).

3.3 Recommended Approach

REST should be used for the reading and writing of request data because it is simple, testable and aligned with the team's existing level of experience. Short-interval client polling should be used to allow the Requester Portal to detect changes rather than introducing a WebSocket layer at this stage.

Polling is considered proportionate because it can satisfy NFR-002 without requiring additional real-time infrastructure. Its failure behaviour is also relatively straightforward: a failed or missed request results in a delayed refresh rather than a persistent connection becoming unavailable.

Push-based communication through WebSockets or Server-Sent Events can be considered as a future improvement if system requirements or team capacity justify it. It is not required as a day-one implementation.

This decision should extend D-002 in the Engineering Decision Log by explicitly recording the client synchronisation mechanism. A short Architecture Decision Record (ADR) should also document the decision. The approach should be reflected in the Requirements Traceability Matrix (RTM) Design/Architecture column against FR-003, FR-007 and NFR-002.

Task 4

Collaborative Engineering and CI Controls

4.1 Source Control Management and Branching

Git provides the mechanism for tracking changes to the project files, while Source Control Management (SCM) is the broader process used to maintain consistency between code, configuration and project versions. This is particularly relevant to RSK-006, where configuration differences between development, staging and production environments could result in inconsistent behaviour.

The project already uses protected main-branch development with a two-reviewer approval model. The branching strategy should therefore keep changes small and reviews manageable.

A trunk-based approach is suitable for the three-person team because it encourages short-lived branches and frequent integration into the main branch. This reduces the likelihood of long-running branches becoming out of sync and helps limit merge conflicts.

Gitflow was considered as an alternative, but its use of multiple long-lived branches would introduce additional process overhead for a small project. A simpler trunk-based approach is therefore more appropriate for CivicConnect (Atlassian, n.d.; AWS, n.d.).

The two-reviewer requirement also creates a potential risk for a three-person team because reviewers could approve changes without sufficient scrutiny simply to keep the work moving. This can be reduced by keeping pull requests small and focused on one feature, requirement or project artefact at a time.

4.2 Continuous Integration and Quality Gates

Git and branch protection help control how changes are merged, but they do not confirm that the application still builds or functions correctly. Continuous Integration (CI) addresses this by automatically building and testing changes whenever they are integrated.

For CivicConnect, CI should run when a pull request targets the main branch. The build should use a clean checkout, a repeatable command and locked dependencies so that it does not depend on an individual developer's local environment.

Due to RSK-001, the initial CI quality gates should focus on the checks that provide the most value without introducing unnecessary complexity. The application should successfully compile or build, and unit tests should pass before a pull request can be merged. These should be configured as required and blocking checks.

Linting and static analysis can initially be configured as warning or non-blocking checks. Once the team is comfortable with the tooling and the rules are producing useful results, they can be promoted to required checks.

CI also provides a control for RSK-005. Secrets required by GitHub Actions should be stored using encrypted repository or environment secrets rather than being committed to configuration files. Dependency monitoring and secret scanning should also be enabled where available. CI results should remain visible on the pull request so reviewers can see whether the automated checks passed.

Automation should handle repeatable mechanical checks, while reviewers remain responsible for judging design decisions, requirement coverage and overall correctness.

4.3 Recommended Approach

CivicConnect should use a lightweight GitHub Actions CI workflow that runs for pull requests targeting the main branch. The initial required checks should be the application build and unit tests.

Linting and static analysis should initially remain non-blocking and can be made mandatory once the team has established suitable rules. Dependabot and secret scanning should also be enabled to help identify dependency and credential risks.

The approach should be reflected in the project's branch protection rules, the PED CI and quality section, and the RTM Issue/PR and Test columns during M2 to M4.

This provides a lightweight engineering process that combines protected main-branch development, peer review and automated checks without introducing unnecessary infrastructure or process overhead.

Task 5

A2 finding	Evidence/alternative considered	Recommendation	Project decision it should inform	Expected PED/ADR/RTM/application evidence
Design problem 1	Request lifecycle carries state-dependent behavior across statuses. Alternatives compared: Centralized conditional - enum plus status checks in one service vs State-oriented design - one class per lifecycle state, evaluated against coupling, cohesion, OCP, extensibility, testability and over-engineering risk.	Use the centralized conditional approach while the lifecycle stays small and stable, move to a state-oriented design only if state-dependent behavior grows substantially and is expected to keep evolving. Final choice confirmed at M2 against the team's actual lifecycle complexity.	M2 design decision on how request-lifecycle status behavior is structured.	ADR recording the chosen approach, the rejected alternative and the trigger for revising it, in the PED v2.0 design section.
Design problem 2	Category-specific request creation and validation needs different fields and rules per category. Alternatives compared: Factory-based creation vs Strategy-based category handlers, evaluated against OCP support, cohesion, testability, additional abstraction and over-engineering risk, with empirical caution that pattern use alone is not evidence of better maintainability.	Factory-based creation where categories mainly differ in construction, Strategy-based handlers where categories are found to have substantially different, independently evolving rules. Final choice confirmed at M2 once category rules are known.	M2 decision on how category-specific request creation and validation is structured.	ADR recording the chosen approach, the rejected alternative and the trigger for revising it, in the PED v2.0 design section.

A2 finding	Evidence/alternative considered	Recommendation	Project decision it should inform	Expected PED/ADR/RTM/application evidence
Persistence/Data	Assigning a request saves five things (owner, status, history, audit, requester message). A partial save or two staff assigning at once breaks NFR-005 and NFR-007.	Approaches compared: A (backend only), B (PostgreSQL stored procedure), C (layered). Concurrency options: conditional UPDATE, version number, SELECT ... FOR UPDATE, Serializable. Caching versus stale data also considered. Approach C: one PostgreSQL transaction run by the backend; conditional UPDATE to claim a request; version column for other edits; PostgreSQL constraints; insert-only history; no caching in M2.	D-003 (PostgreSQL); new persistence ADR; M2 data model; Risk Register	ADR and data model in PED v2.0; new risks (partial save, double ownership); RTM links for FR-011–FR-013, NFR-005, NFR-007; M3 rollback and concurrency tests
Integration/API	The Requester Portal must learn of a staff-driven status change within 15 seconds without exposing another requester's data or duplicating business logic across UIs. Alternatives compared: REST with client polling vs REST with push, evaluated against coupling, NFR-002 fit, failure behavior, testability and team-unfamiliarity risk, Richardson Maturity Model consulted for the appropriate REST maturity level.	REST with short-interval client polling for M2, push-based synchronization deferred as a future improvement if requirements or capacity justify it.	M2 decision extending D-002 with the client synchronization mechanism	ADR documenting the polling decision, RTM Architecture links for FR-003, FR-007, NFR-002.
SCM/CI	Uncontrolled integration by a three-person team risks merge conflicts and configuration drift across environments. Trunk-based development compared against Gitflow for branching, CI checks compared for blocking vs warning status.	Trunk-based development with protected main and two-reviewer approval, kept to small focused pull requests, lightweight GitHub Actions CI requiring build and unit tests as blocking checks, with static analysis initially non-blocking, secret scanning enabled.	Team Working Agreement, GitHub branch-protection/governance and the M2–M3 CI/quality-gate setup.	Branch protection rules; PED CI/quality section, RTM Issue and Test columns M2–M4, CI status visible on pull requests

References

- Almadi, S.H., Hooshyar, D. and Ahmad, R.B., 2021. Bad smells of gang of four design patterns: a decade systematic literature review. *Sustainability*, 13(18), p.10256.
- Atlassian (n.d.) *Trunk-based Development*. Available at: <https://www.atlassian.com/continuous-delivery/continuous-integration/trunk-based-development> (Accessed: 12 September 2026).

- AWS (n.d.) *Advantages and disadvantages of the Trunk strategy*. Available at: <https://docs.aws.amazon.com/prescriptive-guidance/latest/choosing-git-branch-approach/advantages-and-disadvantages-of-the-trunk-strategy.html> (Accessed: 12 September 2026).
- Devopedia (2020) *Richardson Maturity Model*. Available at: <https://devopedia.org/richardson-maturity-model> (Accessed: 12 September 2026).
- Fowler, M. (2006) *Continuous Integration*. Available at: <https://martinfowler.com/articles/continuousIntegration.html> (Accessed: 12 September 2026).
- Microsoft (2025) *Cache-Aside pattern* — Azure Architecture Center. Available at: <https://learn.microsoft.com/en-us/azure/architecture/patterns/cache-aside> (Accessed: 12 September 2026).
- Naghdiipour, A., Hasheminejad, S.M.H. and Barmaki, R.L., 2023. Software design pattern selection approaches: a systematic literature review. *Software: Practice and Experience*, 53(4), pp.1091-1122.
- Nikitin, D., 2023. Specification formalization of state charts for complex system management. *Bulletin of National Technical University "KhPI". Series: System analysis, control and information technologies*, (1 (9)), pp.104-109.
- Plösch, R., Bräuer, J., Körner, C. and Saft, M., 2016. Measuring, assessing and improving software quality based on object-oriented design principles. *Open Computer Science*, 6(1), pp.187-207.
- PostgreSQL Global Development Group (2026a) *PostgreSQL 18 documentation: Transaction isolation*. Available at: <https://www.postgresql.org/docs/current/transaction-iso.html> (Accessed: 12 September 2026).
- RESTful API (n.d.) *Richardson Maturity Model*. Available at: <https://restfulapi.net/richardson-maturity-model/> (Accessed: 12 September 2026).
- Silva, G., Andrade, V., Ré, R. and Meneses, R., 2021, September. A quasi-experiment to investigating the impact of the strategy design pattern on maintainability. In *Proceedings of the XXXV Brazilian Symposium on Software Engineering* (pp. 105-114).
- Stevenson, J. and Wood, M., 2018. Recognizing object-oriented software design quality: a practitioner-based questionnaire survey. *Software Quality Journal*, 26(2), pp.321-365.
- Tiwari, S. and Rathore, S.S., 2018, February. Coupling and cohesion metrics for object-oriented software: A systematic mapping study. In *Proceedings of the 11th Innovations in Software Engineering Conference* (pp. 1-11).
- Wedyan, F. and Abufakher, S., 2020. Impact of design patterns on software quality: a systematic literature review. *IET Software*, 14(1), pp.1-17.